

Security Scan Report

Target: C:\Users\Purushottam\OneDrive\Desktop\CyberScurity Project - Copy\Test

Scan Date: August 26, 2026 at 22:00 UTC

Scan ID: d56855f0-df2f-4ffb-824e-fa8f994d4ca7

1 CRITICAL

4 HIGH

5 MEDIUM

6 LOW

Executive Summary

Security scan of 'C:\Users\Purushottam\OneDrive\Desktop\CyberSceurity Project - Copy\Test' identified 16 raw findings, reduced to 16 unique issues after deduplication. ■■ 1 CRITICAL issue(s) require immediate attention and should be fixed before the next deployment. ■ 4 HIGH-severity issue(s) should be prioritized for remediation within the next 48 hours. 5 MEDIUM-priority issue(s) should be addressed in the current development sprint. 6 LOW-priority issue(s) have been logged for backlog tracking.

Scan Statistics

Metric	Value
Total Raw Findings	16
After Deduplication	16
Critical Findings	1
High Findings	4
Medium Findings	5
Low Findings	6
Scan Duration	30.7s

Detailed Findings

[HIGH] #1: Sql Injection in C:\Users\Purushottam\OneDrive\Desktop\CyberScurity Project - Copy\Test\vuln_bank_lab\app.py:70

Risk Score	0.6435
Affected Location	C:\Users\Purushottam\OneDrive\Desktop\CyberScurity Project - Copy\Test\vuln_bank_lab\app.py:70
Vulnerability Type	SQL_INJECTION
CWE ID	CWE-89
Detected By	semgrep

Explanation
SQL Injection occurs when user-controlled input is directly interpolated into SQL queries, allowing an attacker to alter query logic. Found in app.py. Potential SQL injection via dynamic query construction.

Code or HTTP Excerpt

```
68 |         email = request.form.get("email", "")
69 |         password = request.form.get("password", "")
>>> 70 |         query = f"SELECT * FROM users WHERE email = '{email}' AND password = '{password}'"
71 |         try:
72 |             conn = connect()
```

Business Impact
An attacker could read, modify, or delete user credentials and passwords, potentially exfiltrating the entire database including credentials and PII.

Fix Recommendation
Best Fix: Use parameterized/prepared queries instead of string concatenation. Pass user input as bind parameters, not as part of the query string.

[CRITICAL] #2: Rce in C:\Users\Purushottam\OneDrive\Desktop\CyberScurity Project - Copy\Test\vuln_bank_lab\app.py:282

Risk Score	0.5817
Affected Location	C:\Users\Purushottam\OneDrive\Desktop\CyberScurity Project - Copy\Test\vuln_bank_lab\app.py:282
Vulnerability Type	RCE
CWE ID	CWE-94
Detected By	semgrep

Explanation
Remote Code Execution allows an attacker to execute arbitrary system commands on the server through user-controlled input. Found in app.py. Potential remote code execution or unsafe deserialization.

Code or HTTP Excerpt

```
280 |         try:
281 |             decoded = base64.b64decode(payload)
>>> 282 |             session_object = pickle.loads(decoded)
283 |             return jsonify({"status": "restored", "object": repr(session_object)})
284 |         except Exception as exc:
```

Business Impact

An attacker could take complete control of user session data, access all data, install malware, and pivot to other infrastructure.

Fix Recommendation

Best Fix: Remove `exec()` calls on user-controlled input. Use a sandboxed environment or whitelist of allowed operations instead.

[HIGH] #3: Ssrf in C:\Users\Purushottam\OneDrive\Desktop\CyberScurity Project - Copy\Test\vuln_bank_lab\app.py:248

Risk Score	0.5227
Affected Location	C:\Users\Purushottam\OneDrive\Desktop\CyberScurity Project - Copy\Test\vuln_bank_lab\app.py:248
Vulnerability Type	SSRF
CWE ID	CWE-918
Detected By	semgrep

Explanation

Server-Side Request Forgery (SSRF) lets attackers make the server send HTTP requests to arbitrary URLs, including internal services. Found in `app.py`. Potential SSRF via user-controlled outbound request.

Code or HTTP Excerpt

```
246 | def fetch_remote():
247 |     target = request.args.get("url", "http://127.0.0.1:5000/")
>>> 248 |     with urllib.request.urlopen(target, timeout=3) as handle:
249 |         body = handle.read(3000).decode("utf-8", errors="replace")
250 |     return Response(body, mimetype="text/plain")
```

Business Impact

An attacker could access user accounts and personal data from within the server's network, potentially reading cloud metadata, internal APIs, or admin panels.

Fix Recommendation

Best Fix: Replace `urllib.request.urlopen()` with a validated request: parse the URL, check the hostname is not in private IP ranges (10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16, 169.254.169.254), and whitelist allowed domains.

[MEDIUM] #4: Hardcoded Secret in C:\Users\Purushottam\OneDrive\Desktop\CyberScurity Project - Copy\Test\vuln_bank_lab\app.py:70

Risk Score	0.5117
Affected Location	C:\Users\Purushottam\OneDrive\Desktop\CyberScurity Project - Copy\Test\vuln_bank_lab\app.py:70
Vulnerability Type	HARDCODED_SECRET
CWE ID	CWE-798
Detected By	semgrep

Explanation

Hardcoded credentials, API keys, or tokens in source code can be extracted by anyone with repository access. Found in app.py. Hardcoded secret detected in source code.

Code or HTTP Excerpt

```
68 |         email = request.form.get("email", "")
69 |         password = request.form.get("password", "")
>>> 70 |         query = f"SELECT * FROM users WHERE email = '{email}' AND password = '{password}'"
71 |         try:
72 |             conn = connect()
```

Business Impact

The exposed credentials could grant unauthorized access to user credentials and passwords, enabling data theft or service abuse.

Fix Recommendation

Best Fix: Move the password to an environment variable. Use `os.environ.get('DB_PASSWORD')` instead of the literal string. Rotate the exposed password immediately.

[MEDIUM] #5: Xxe in C:\Users\Purushottam\OneDrive\Desktop\CyberScurity Project - Copy\Test\vuln_bank_lab\app.py:292

Risk Score	0.5023
Affected Location	C:\Users\Purushottam\OneDrive\Desktop\CyberScurity Project - Copy\Test\vuln_bank_lab\app.py:292
Vulnerability Type	XXE
CWE ID	CWE-611
Detected By	semgrep

Explanation

XML External Entity (XXE) injection allows attackers to read server files, make outbound requests, or cause denial of service through crafted XML input. Found in app.py. Potential XXE due to unsafe XML parsing or external entity processing.

Code or HTTP Excerpt

```
290 |         raw_xml = request.get_data(as_text=True) or request.form.get("xml", "")
291 |         leaked = {}
>>> 292 |         entity_matches = re.findall(r'<!ENTITY\s+(\w+)\s+SYSTEM\s+["\']file://[^\s"\']+[^\s"\']>', raw_xml)
293 |         for name, file_path in entity_matches:
294 |             try:
```

Business Impact

An attacker could read files from user accounts and personal data, make outbound requests to internal services, or cause denial of service.

Fix Recommendation

Best Fix: Use `defusedxml` instead of the standard `xml` library. Disable external entity resolution and DTD processing.

[HIGH] #6: Sql Injection in C:\Users\Purushottam\OneDrive\Desktop\CyberScurity Project - Copy\Test\vuln_bank_lab\db.py:99

Risk Score	0.4950
Affected Location	C:\Users\Purushottam\OneDrive\Desktop\CyberScurity Project - Copy\Test\vuln_bank_lab\db.py:99
Vulnerability Type	SQL_INJECTION
CWE ID	CWE-89
Detected By	semgrep

Explanation

SQL Injection occurs when user-controlled input is directly interpolated into SQL queries, allowing an attacker to alter query logic. Found in db.py. Potential SQL injection via dynamic query construction.

Code or HTTP Excerpt

```
97 | )
98 |
>>> 99 | if cur.execute("SELECT COUNT(*) AS c FROM users").fetchone()["c"] == 0:
100 |     cur.executemany("INSERT INTO users VALUES (?, ?, ?, ?, ?, ?)", USERS)
101 |     cur.executemany("INSERT INTO accounts VALUES (?, ?, ?, ?, ?, ?)", ACCOUNTS)
```

Business Impact

An attacker could read, modify, or delete user accounts and personal data, potentially exfiltrating the entire database including credentials and PII.

Fix Recommendation

Best Fix: Use parameterized/prepared queries instead of string concatenation. Pass user input as bind parameters, not as part of the query string.

[MEDIUM] #7: Xxe in C:\Users\Purushottam\OneDrive\Desktop\CyberScurity Project - Copy\Test\vuln_bank_lab\templates\base.html:1

Risk Score	0.4444
Affected Location	C:\Users\Purushottam\OneDrive\Desktop\CyberScurity Project - Copy\Test\vuln_bank_lab\templates
Vulnerability Type	XXE
CWE ID	CWE-611
Detected By	semgrep

Explanation

XML External Entity (XXE) injection allows attackers to read server files, make outbound requests, or cause denial of service through crafted XML input. Found in base.html. Potential XXE due to unsafe XML parsing or external entity processing.

Code or HTTP Excerpt

```
>>> 1 | <!doctype html>
      2 | <html lang="en">
      3 | <head>
```

Business Impact

An attacker could read files from user accounts and personal data, make outbound requests to internal services, or cause denial of service.

Fix Recommendation

Best Fix: Use defusedxml instead of the standard xml library. Disable external entity resolution and DTD processing.

[HIGH] #8: Missing Auth in
C:\Users\Purushottam\OneDrive\Desktop\CyberScurity Project -
Copy\Test\vuln_bank_lab\app.py:23

Risk Score	0.4418
Affected Location	C:\Users\Purushottam\OneDrive\Desktop\CyberScurity Project - Copy\Test\vuln_bank_lab\app.py:23
Vulnerability Type	MISSING_AUTH
CWE ID	CWE-306
Detected By	semgrep

Explanation
This endpoint or function lacks authentication, allowing unauthenticated users to access protected resources. Found in app.py. Potential insecure configuration or missing auth controls.

Code or HTTP Excerpt

```
21 | app.config.update(  
22 |     SECRET_KEY="bank-lab-secret-2026",  
>>> 23 |     SESSION_COOKIE_HTTPONLY=False,  
24 |     SESSION_COOKIE_SECURE=False,  
25 |     SESSION_COOKIE_SAMESITE=None,
```

Business Impact
Unauthenticated users could access user session data, potentially viewing or modifying sensitive data without authorization.

Fix Recommendation
Best Fix: Add authentication middleware or decorators to all sensitive endpoints. Verify user identity before serving protected data.

[MEDIUM] #9: Insecure Cookie in
C:\Users\Purushottam\OneDrive\Desktop\CyberScurity Project -
Copy\Test\vuln_bank_lab\app.py:77

Risk Score	0.4416
Affected Location	C:\Users\Purushottam\OneDrive\Desktop\CyberScurity Project - Copy\Test\vuln_bank_lab\app.py:77
Vulnerability Type	INSECURE_COOKIE
CWE ID	CWE-614
Detected By	semgrep

Explanation
One or more cookies are missing important security flags, making them vulnerable to theft or misuse. Found in app.py. Insecure cookie configuration with missing security attributes.

Code or HTTP Excerpt

```
75 |         if user:  
76 |             response = redirect(url_for("dashboard"))  
>>> 77 |             response.set_cookie("bank_uid", str(user["id"]), httponly=False, secure=False)
```

```
78 |         return response
79 |         error = "Invalid credentials"
```

Business Impact

The session cookies cookie could be stolen via XSS or transmitted over insecure connections, enabling session hijacking.

Fix Recommendation

Best Fix: Set Secure, HttpOnly, and SameSite flags on all sensitive cookies.

[MEDIUM] #10: Xss in C:\Users\Purushottam\OneDrive\Desktop\CyberSceurity Project - Copy\Test\vuln_bank_lab\templates\admin.html:22

Risk Score	0.4361
Affected Location	C:\Users\Purushottam\OneDrive\Desktop\CyberSceurity Project - Copy\Test\vuln_bank_lab\templates
Vulnerability Type	XSS
CWE ID	CWE-79
Detected By	semgrep

Explanation

Cross-Site Scripting (XSS) allows attackers to inject malicious scripts into web pages viewed by other users. Found in admin.html. Potential XSS via unsafe output handling.

Code or HTTP Excerpt

```
>>> 20 |         <button class="btn primary" type="submit">Run</button>
      21 |         </form>
      22 |         <pre class="console">{{ command_output|safe }}</pre>
      23 |     </div>
      24 | <div class="card">
```

Business Impact

An attacker could steal session tokens from the admin panel, redirect users to phishing pages, or perform actions as the victim.

Fix Recommendation

Best Fix: Apply context-aware output encoding before rendering user content. Use your framework's built-in auto-escaping.

[LOW] #11: Idor in C:\Users\Purushottam\OneDrive\Desktop\CyberSceurity Project - Copy\Test\vuln_bank_lab\app.py:44

Risk Score	0.3936
Affected Location	C:\Users\Purushottam\OneDrive\Desktop\CyberSceurity Project - Copy\Test\vuln_bank_lab\app.py:44
Vulnerability Type	IDOR
CWE ID	CWE-639
Detected By	semgrep

Explanation

Insecure Direct Object Reference (IDOR) allows users to access other users' data by changing predictable resource identifiers. Found in app.py. Potential IDOR due to direct object reference.

Code or HTTP Excerpt

```
42 | @app.before_request
43 | def load_current_user():
>>> 44 |     user_id = request.cookies.get("bank_uid")
45 |     g.current_user = None
46 |     if user_id and user_id.isdigit():
```

Business Impact

An attacker could access or modify other users' session cookies by incrementing or guessing resource identifiers.

Fix Recommendation

Best Fix: Add authorization checks that verify the requesting user owns or has permission to access the requested resource. Use UUIDs instead of sequential integer IDs to make enumeration harder.

[LOW] #12: Xss in C:\Users\Purushottam\OneDrive\Desktop\CyberScurity Project - Copy\Test\vuln_bank_lab\templates\account.html:24

Risk Score	0.3626
Affected Location	C:\Users\Purushottam\OneDrive\Desktop\CyberScurity Project - Copy\Test\vuln_bank_lab\templates
Vulnerability Type	XSS
CWE ID	CWE-79
Detected By	semgrep

Explanation

Cross-Site Scripting (XSS) allows attackers to inject malicious scripts into web pages viewed by other users. Found in account.html. Potential XSS via unsafe output handling.

Code or HTTP Excerpt

```
22 |     {% for tx in transactions %}
23 |         <tr>
>>> 24 |             <td>{{ tx.counterparty|safe }}</td>
25 |             <td>{{ '%.2f'|format(tx.amount) }}</td>
26 |             <td>{{ tx.note|safe }}</td>
```

Business Impact

An attacker could steal session tokens from user accounts and personal data, redirect users to phishing pages, or perform actions as the victim.

Fix Recommendation

Best Fix: Apply context-aware output encoding before rendering user content. Use your framework's built-in auto-escaping.

[LOW] #13: Xss in C:\Users\Purushottam\OneDrive\Desktop\CyberScurity Project - Copy\Test\vuln_bank_lab\templates\dashboard.html:61

Risk Score	0.3626
------------	--------

Affected Location	C:\Users\Purushottam\OneDrive\Desktop\CyberScurity Project - Copy\Test\vuln_bank_lab\templates
Vulnerability Type	XSS
CWE ID	CWE-79
Detected By	semgrep

Explanation
Cross-Site Scripting (XSS) allows attackers to inject malicious scripts into web pages viewed by other users. Found in dashboard.html. Potential XSS via unsafe output handling.

Code or HTTP Excerpt

```
59 |         <td>{{ tx.counterparty }}</td>
60 |         <td>{{ '%.2f'|format(tx.amount) }}</td>
>>> 61 |         <td>{{ tx.note|safe }}</td>
62 |     </tr>
63 |     {% endfor %}
```

Business Impact
An attacker could steal session tokens from user accounts and personal data, redirect users to phishing pages, or perform actions as the victim.

Fix Recommendation
Best Fix: Apply context-aware output encoding before rendering user content. Use your framework's built-in auto-escaping.

[LOW] #14: Xss in C:\Users\Purushottam\OneDrive\Desktop\CyberScurity Project - Copy\Test\vuln_bank_lab\templates\reflect.html:7

Risk Score	0.3626
Affected Location	C:\Users\Purushottam\OneDrive\Desktop\CyberScurity Project - Copy\Test\vuln_bank_lab\templates
Vulnerability Type	XSS
CWE ID	CWE-79
Detected By	semgrep

Explanation
Cross-Site Scripting (XSS) allows attackers to inject malicious scripts into web pages viewed by other users. Found in reflect.html. Potential XSS via unsafe output handling.

Code or HTTP Excerpt

```
5 | <h1>Customer Notice Board</h1>
6 | <p class="subtle">This page intentionally renders query params unsafely.</p>
>>> 7 | <div class="alert">Hello {{ name|safe }}, message: {{ message|safe }}</div>
8 | <form method="get" class="form inline" action="/reflect">
9 |     <label>Name<input type="text" name="name" value="{{ name|safe }}"></label>
```

Business Impact
An attacker could steal session tokens from user accounts and personal data, redirect users to phishing pages, or perform actions as the victim.

Fix Recommendation
Best Fix: Apply context-aware output encoding before rendering user content. Use your framework's built-in auto-escaping.

[LOW] #15: Xss in C:\Users\Purushottam\OneDrive\Desktop\CyberScurity Project - Copy\Test\vuln_bank_lab\templates\support.html:23

Risk Score	0.3626
Affected Location	C:\Users\Purushottam\OneDrive\Desktop\CyberScurity Project - Copy\Test\vuln_bank_lab\templates
Vulnerability Type	XSS
CWE ID	CWE-79
Detected By	semgrep

Explanation

Cross-Site Scripting (XSS) allows attackers to inject malicious scripts into web pages viewed by other users. Found in support.html. Potential XSS via unsafe output handling.

Code or HTTP Excerpt

```
21 |         <td>{{ ticket.name }}</td>
22 |         <td>{{ ticket.subject }}</td>
>>> 23 |         <td>{{ ticket.body|safe }}</td>
24 |     </tr>
25 |     {% endfor %}
```

Business Impact

An attacker could steal session tokens from user accounts and personal data, redirect users to phishing pages, or perform actions as the victim.

Fix Recommendation

Best Fix: Apply context-aware output encoding before rendering user content. Use your framework's built-in auto-escaping.

[LOW] #16: Open Redirect in C:\Users\Purushottam\OneDrive\Desktop\CyberScurity Project - Copy\Test\vuln_bank_lab\app.py:88

Risk Score	0.3456
Affected Location	C:\Users\Purushottam\OneDrive\Desktop\CyberScurity Project - Copy\Test\vuln_bank_lab\app.py:88
Vulnerability Type	OPEN_REDIRECT
CWE ID	CWE-601
Detected By	semgrep

Explanation

Open redirect allows attackers to redirect users from your trusted domain to a malicious site via a URL parameter. Found in app.py. Potential open redirect using attacker-controlled redirect target.

Code or HTTP Excerpt

```
86 | def logout():
87 |     next_url = request.args.get("next", url_for("index"))
>>> 88 |     response = redirect(next_url)
89 |     response.delete_cookie("bank_uid")
90 |     return response
```

Business Impact

Attackers could send users convincing phishing links via session cookies that appear to originate from your trusted domain.

Fix Recommendation

Best Fix: Validate redirect URLs against a whitelist of allowed domains. Use relative paths for internal redirects. Reject external URLs.

Appendix

Deduplication Statistics

Total raw findings: 16 | After deduplication: 16 | Reduction: 0%

Scan Metadata

Scan ID	d56855f0-df2f-4ffb-824e-fa8f994d4ca7
Target	C:\Users\Purushottam\OneDrive\Desktop\CyberScurity Project - Copy\Test
Timestamp	2026-08-26T22:00:23.137252+00:00
Duration	30.7s
Scanners Used	semgrep
Scanner Versions	semgrep=unknown

Raw Findings Data (JSON)

```
[
  {
    "id": "077cf5f3-70d1-473d-8672-263e03448ef7",
    "title": "Sql Injection in C:\\Users\\Purushottam\\OneDrive\\Desktop\\CyberScurity Project - Copy\\Test",
    "file": "C:\\Users\\Purushottam\\OneDrive\\Desktop\\CyberScurity Project - Copy\\Test\\vuln_bank_lab\\",
    "line_start": 70,
    "line_end": 70,
    "severity": "HIGH",
    "vuln_type": "SQL_INJECTION",
    "description": "Potential SQL injection via dynamic query construction.",
    "rule_id": "fallback-sql_injection",
    "confidence": 0.95,
    "cwe_id": "CWE-89",
    "cvss_score": null,
    "code_snippet": "
        68 |         email = request.form.get(\"email\", \"\")\n
        69 |         passwo",
    "sources": [
      "semgrep"
    ],
    "risk_score": 0.6435,
    "risk_category": "HIGH",
    "category_source": "score",
    "category_reason": "score-threshold",
    "llm_explanation": "SQL Injection occurs when user-controlled input is directly interpolated into SQL qu",
    "fix_suggestion": "Use parameterized/prepared queries instead of string concatenation. Pass user input a",
    "business_impact": "An attacker could read, modify, or delete user credentials and passwords, potential",
    "attack_path": "UNKNOWN: deterministic fallback has no runtime proof.",
    "next_test": "Run one authorized, read-only validation test and stop if it changes state or exposes rea",
    "raw": {
      "pattern": "(execute\\(|cursor\\.execute\\(|query\\s*=\\s*f?['\\\"].*(SELECT|INSERT|UPDATE|DELETE).*[",
      "match": "query = f\"SELECT * FROM users WHERE email = '{email}' AND password = '{password}'\"",
      "file": "C:\\Users\\Purushottam\\OneDrive\\Desktop\\CyberScurity Project - Copy\\Test\\vuln_bank_lab"
    }
  },
  {
    "id": "378dd6da-675f-4977-bb94-987adc8c92d8",
    "title": "Rce in C:\\Users\\Purushottam\\OneDrive\\Desktop\\CyberScurity Project - Copy\\Test\\vuln_bar",
    "file": "C:\\Users\\Purushottam\\OneDrive\\Desktop\\CyberScurity Project - Copy\\Test\\vuln_bank_lab\\",
    "line_start": 282,
    "line_end": 282,
```

```

"severity": "HIGH",
"vuln_type": "RCE",
"description": "Potential remote code execution or unsafe deserialization.",
"rule_id": "fallback-rce",
"confidence": 0.93,
"cwe_id": "CWE-94",
"cvss_score": null,
"code_snippet": "        280 |         try:\n        281 |             decoded = base64.b64decode(payload)\n>>> 282
"sources": [
    "semgrep"
],
"risk_score": 0.5817,
"risk_category": "CRITICAL",
"category_source": "vuln_floor",
"category_reason": "minimum floor for RCE",
"llm_explanation": "Remote Code Execution allows an attacker to execute arbitrary system commands on the",
"fix_suggestion": "Remove exec() calls on user-controlled input. Use a sandboxed environment or whitelisting",
"business_impact": "An attacker could take complete control of user session data, access all data, insta",
"attack_path": "UNKNOWN: deterministic fallback has no runtime proof.",
"next_test": "Run one authorized, read-only validation test and stop if it changes state or exposes rea",
"raw": {
    "pattern": "(eval\\(|exec\\(|os\\.system\\(|subprocess\\.\\.?:run|call|Popen)\\(|\\.shell\\|s*=\\|s*True|pi",
    "match": "pickle.loads(",
    "file": "C:\\Users\\Purushottam\\OneDrive\\Desktop\\CyberScurity Project - Copy\\Test\\vuln_bank_lab\\
}
},
{
    "id": "aef5a01c-c95c-40a4-bcfd-9481a5d67cf8",
    "title": "Ssrf in C:\\Users\\Purushottam\\OneDrive\\Desktop\\CyberScurity Project - Copy\\Test\\vuln_bank_lab\\",
    "file": "C:\\Users\\Purushottam\\OneDrive\\Desktop\\CyberScurity Project - Copy\\Test\\vuln_bank_lab\\",
    "line_start": 248,
    "line_end": 248,
    "severity": "HIGH",
    "vuln_type": "SSRF",
    "description": "Potential SSRF via user-controlled outbound request.",
    "rule_id": "fallback-ssrf",
    "confidence": 0.84,
    "cwe_id": "CWE-918",
    "cvss_score": null,
    "code_snippet": "        246 | def fetch_remote():\n        247 |         target = request.args.get(\"url\", \"ht",
    "sources": [
        "semgrep"
    ],
    "risk_score": 0.5227,
    "risk_category": "HIGH",
    "category_source": "vuln_floor",
    "category_reason": "minimum floor for SSRF",
    "llm_explanation": "Server-Side Request Forgery (SSRF) lets attackers make the server send HTTP requests",
    "fix_suggestion": "Replace urllib.request.urlopen() with a validated request: parse the URL, check the h",
    "business_impact": "An attacker could access user accounts and personal data from within the server's ne",
    "attack_path": "UNKNOWN: deterministic fallback has no runtime proof.",
    "next_test": "Run one authorized, read-only validation test and stop if it changes state or exposes rea",
    "raw": {
        "pattern": "(urlopen\\(|requests\\.\\.?:get|post|put|delete)\\(|\\.request\\.\\.httpx\\.\\.?:get|post)\\(|\\.re",
        "match": "urlopen(",
        "file": "C:\\Users\\Purushottam\\OneDrive\\Desktop\\CyberScurity Project - Copy\\Test\\vuln_bank_lab\\
    }
},
{
    "id": "0eeaeecd4-d813-4e6f-9254-6893b2105f1e",
    "title": "Hardcoded Secret in C:\\Users\\Purushottam\\OneDrive\\Desktop\\CyberScurity Project - Copy\\",
    "file": "C:\\Users\\Purushottam\\OneDrive\\Desktop\\CyberScurity Project - Copy\\Test\\vuln_bank_lab\\",
    "line_start": 70,
    "line_end": 70,
    "severity": "MEDIUM",
    "vuln_type": "HARDCODED_SECRET",
    "description": "Hardcoded secret detected in source code.",
    "rule_id": "fallback-hardcoded_secret",
    "confidence": 0.92,
    "cwe_id": "CWE-798",
    "cvss_score": null,

```

```

"code_snippet": "      68 |          email = request.form.get("email", "")\n      69 |          passwo\n"sources": [\n    "semgrep"\n],\n"risk_score": 0.5117,\n"risk_category": "MEDIUM",\n"category_source": "score",\n"category_reason": "score-threshold",\n"llm_explanation": "Hardcoded credentials, API keys, or tokens in source code can be extracted by anyone",\n"fix_suggestion": "Move the password to an environment variable. Use os.environ.get('DB_PASSWORD') inste",\n"business_impact": "The exposed credentials could grant unauthorized access to user credentials and pass",\n"attack_path": "UNKNOWN: deterministic fallback has no runtime proof.",\n"next_test": "Run one authorized, read-only validation test and stop if it changes state or exposes rea",\n"raw": {\n    "pattern": "(password\\s*=\\s*['\\\"\\\"'].+['\\\"\\\"']|api[_-]?key\\s*=\\s*['\\\"\\\"'].+['\\\"\\\"']|secret\\s*=\\s*['\\\"\\\"']",\n    "match": "password = '{password}'\"",\n    "file": "C:\\Users\\Purushottam\\OneDrive\\Desktop\\CyberScurity Project - Copy\\Test\\vuln_bank_lab\\",\n}\n},\n{\n    "id": "6247d3ef-7180-4a65-96ed-c9547e9c49a7",\n    "title": "Xxe in C:\\Users\\Purushottam\\OneDrive\\Desktop\\CyberScurity Project - Copy\\Test\\vuln_bank_lab\\",\n    "file": "C:\\Users\\Purushottam\\OneDrive\\Desktop\\CyberScurity Project - Copy\\Test\\vuln_bank_lab\\",\n    "line_start": 292,\n    "line_end": 292,\n    "severity": "HIGH",\n    "vuln_type": "XXE",\n    "description": "Potential XXE due to unsafe XML parsing or external entity processing.",\n    "rule_id": "fallback-xxe",\n    "confidence": 0.83,\n    "cwe_id": "CWE-611",\n    "cvss_score": null,\n    "code_snippet": "      290 |          raw_xml = request.get_data(as_text=True) or request.form.get("xml", "")\n"sources": [\n    "semgrep"\n],\n"risk_score": 0.5023,\n"risk_category": "MEDIUM",\n"category_source": "score",\n"category_reason": "score-threshold",\n"llm_explanation": "XML External Entity (XXE) injection allows attackers to read server files, make outb",\n"fix_suggestion": "Use defusedxml instead of the standard xml library. Disable external entity resolution",\n"business_impact": "An attacker could read files from user accounts and personal data, make outbound re",\n"attack_path": "UNKNOWN: deterministic fallback has no runtime proof.",\n"next_test": "Run one authorized, read-only validation test and stop if it changes state or exposes rea",\n"raw": {\n    "pattern": "(xml\\.fromstring\\(|ET\\.fromstring\\(|ElementTree\\.fromstring\\(|<!ENTITY|DOCTYPE|featu",\n    "match": "<!ENTITY",\n    "file": "C:\\Users\\Purushottam\\OneDrive\\Desktop\\CyberScurity Project - Copy\\Test\\vuln_bank_lab\\",\n}\n},\n{\n    "id": "8840ab78-1262-481e-bb1b-002d1f1f90bd",\n    "title": "Sql Injection in C:\\Users\\Purushottam\\OneDrive\\Desktop\\CyberScurity Project - Copy\\Test\\vuln_bank_lab\\",\n    "file": "C:\\Users\\Purushottam\\OneDrive\\Desktop\\CyberScurity Project - Copy\\Test\\vuln_bank_lab\\",\n    "line_start": 99,\n    "line_end": 99,\n    "severity": "HIGH",\n    "vuln_type": "SQL_INJECTION",\n    "description": "Potential SQL injection via dynamic query construction.",\n    "rule_id": "fallback-sql_injection",\n    "confidence": 0.95,\n    "cwe_id": "CWE-89",\n    "cvss_score": null,\n    "code_snippet": "      97 |         )\n      98 |         \n      99 |         if cur.execute("SELECT COUNT(*) AS c FROM\n"sources": [\n    "semgrep"\n],\n"risk_score": 0.495,\n"risk_category": "HIGH",\n"category_source": "vuln_floor",

```

```

"category_reason": "minimum floor for SQL_INJECTION",
"llm_explanation": "SQL Injection occurs when user-controlled input is directly interpolated into SQL qu
"fix_suggestion": "Use parameterized/prepared queries instead of string concatenation. Pass user input a
"business_impact": "An attacker could read, modify, or delete user accounts and personal data, potentia
"attack_path": "UNKNOWN: deterministic fallback has no runtime proof.",
"next_test": "Run one authorized, read-only validation test and stop if it changes state or exposes rea
"raw": {
  "pattern": "(execute\\(|cursor\\.execute\\(|query\\s*=\\s*f?['\\\"].*(SELECT|INSERT|UPDATE|DELETE).*[
  "match": "execute(",
  "file": "C:\\Users\\Purushottam\\OneDrive\\Desktop\\CyberScurity Project - Copy\\Test\\vuln_bank_lab
}
},
{
  "id": "80b96236-e73e-4853-afcd-75d0f79ce381",
  "title": "Xxe in C:\\Users\\Purushottam\\OneDrive\\Desktop\\CyberScurity Project - Copy\\Test\\vuln_ban
  "file": "C:\\Users\\Purushottam\\OneDrive\\Desktop\\CyberScurity Project - Copy\\Test\\vuln_bank_lab\\
  "line_start": 1,
  "line_end": 1,
  "severity": "HIGH",
  "vuln_type": "XXE",
  "description": "Potential XXE due to unsafe XML parsing or external entity processing.",
  "rule_id": "fallback-xxe",
  "confidence": 0.83,
  "cwe_id": "CWE-611",
  "cvss_score": null,
  "code_snippet": ">>> 1 | <!doctype html>\n          2 | <html lang=\"en\">\n          3 | <head>",
  "sources": [
    "semgrep"
  ],
  "risk_score": 0.4444,
  "risk_category": "MEDIUM",
  "category_source": "score",
  "category_reason": "score-threshold",
  "llm_explanation": "XML External Entity (XXE) injection allows attackers to read server files, make out
  "fix_suggestion": "Use defusedxml instead of the standard xml library. Disable external entity resolutio
  "business_impact": "An attacker could read files from user accounts and personal data, make outbound rec
  "attack_path": "UNKNOWN: deterministic fallback has no runtime proof.",
  "next_test": "Run one authorized, read-only validation test and stop if it changes state or exposes rea
  "raw": {
    "pattern": "(xml\\.fromstring\\(|ET\\.fromstring\\(|ElementTree\\.fromstring\\(|<!ENTITY|DOCTYPE|featu
    "match": "doctype",
    "file": "C:\\Users\\Purushottam\\OneDrive\\Desktop\\CyberScurity Project - Copy\\Test\\vuln_bank_lab
  }
},
{
  "id": "37c42a16-6803-435b-a4e9-fcaabf582af1",
  "title": "Missing Auth in C:\\Users\\Purushottam\\OneDrive\\Desktop\\CyberScurity Project - Copy\\Test\\
  "file": "C:\\Users\\Purushottam\\OneDrive\\Desktop\\CyberScurity Project - Copy\\Test\\vuln_bank_lab\\
  "line_start": 23,
  "line_end": 23,
  "severity": "MEDIUM",
  "vuln_type": "MISSING_AUTH",
  "description": "Potential insecure configuration or missing auth controls.",
  "rule_id": "fallback-missing_auth",
  "confidence": 0.7,
  "cwe_id": "CWE-306",
  "cvss_score": null,
  "code_snippet": "21 | app.config.update(\n          22 | SECRET_KEY=\"bank-lab-secret-2026\", \n>>>
  "sources": [
    "semgrep"
  ],
  "risk_score": 0.4418,
  "risk_category": "HIGH",
  "category_source": "vuln_floor",
  "category_reason": "minimum floor for MISSING_AUTH",
  "llm_explanation": "This endpoint or function lacks authentication, allowing unauthenticated users to ac
  "fix_suggestion": "Add authentication middleware or decorators to all sensitive endpoints. Verify user
  "business_impact": "Unauthenticated users could access user session data, potentially viewing or modify
  "attack_path": "UNKNOWN: deterministic fallback has no runtime proof.",
  "next_test": "Run one authorized, read-only validation test and stop if it changes state or exposes rea
  "raw": {

```

```

    "pattern": "(debug\\s*=\\s*True|SESSION_COOKIE_HTTPONLY\\s*=\\s*False|SESSION_COOKIE_SECURE\\s*=\\s*False)",
    "match": "SESSION_COOKIE_HTTPONLY=False",
    "file": "C:\\Users\\Purushottam\\OneDrive\\Desktop\\CyberScurity Project - Copy\\Test\\vuln_bank_lab\\",
  },
  {
    "id": "63d473b0-ee95-4c70-8ead-1bb891e9e6f1",
    "title": "Insecure Cookie in C:\\Users\\Purushottam\\OneDrive\\Desktop\\CyberScurity Project - Copy\\Test\\vuln_bank_lab\\",
    "file": "C:\\Users\\Purushottam\\OneDrive\\Desktop\\CyberScurity Project - Copy\\Test\\vuln_bank_lab\\",
    "line_start": 77,
    "line_end": 77,
    "severity": "MEDIUM",
    "vuln_type": "INSECURE_COOKIE",
    "description": "Insecure cookie configuration with missing security attributes.",
    "rule_id": "fallback-insecure-cookie",
    "confidence": 0.8,
    "cwe_id": "CWE-614",
    "cvss_score": null,
    "code_snippet": "
      75 |
      if user:\\n
      76 |
      response = redirect(url_for(
    "sources": [
      "semgrep"
    ],
    "risk_score": 0.4416,
    "risk_category": "MEDIUM",
    "category_source": "score",
    "category_reason": "score-threshold",
    "llm_explanation": "One or more cookies are missing important security flags, making them vulnerable to",
    "fix_suggestion": "Set Secure, HttpOnly, and SameSite flags on all sensitive cookies.",
    "business_impact": "The session cookies cookie could be stolen via XSS or transmitted over insecure conn",
    "attack_path": "UNKNOWN: deterministic fallback has no runtime proof.",
    "next_test": "Run one authorized, read-only validation test and stop if it changes state or exposes rea",
    "raw": {
      "pattern": "(set_cookie\\(\\.*secure\\s*=\\s*False|set_cookie\\(\\.*httponly\\s*=\\s*False)",
      "match": "set_cookie(\"bank_uid\", str(user[\"id\"]), httponly=False, secure=False)",
      "file": "C:\\Users\\Purushottam\\OneDrive\\Desktop\\CyberScurity Project - Copy\\Test\\vuln_bank_lab\\",
    }
  },
  {
    "id": "7460f715-aa28-4bc4-85d7-b4a541862d74",
    "title": "Xss in C:\\Users\\Purushottam\\OneDrive\\Desktop\\CyberScurity Project - Copy\\Test\\vuln_bank_lab\\",
    "file": "C:\\Users\\Purushottam\\OneDrive\\Desktop\\CyberScurity Project - Copy\\Test\\vuln_bank_lab\\",
    "line_start": 22,
    "line_end": 22,
    "severity": "MEDIUM",
    "vuln_type": "XSS",
    "description": "Potential XSS via unsafe output handling.",
    "rule_id": "fallback-xss",
    "confidence": 0.9,
    "cwe_id": "CWE-79",
    "cvss_score": null,
    "code_snippet": "
      20 |
      <button class=\"btn primary\" type=\"submit\">Run</button>\\n
      21 |
    "sources": [
      "semgrep"
    ],
    "risk_score": 0.4361,
    "risk_category": "MEDIUM",
    "category_source": "score",
    "category_reason": "score-threshold",
    "llm_explanation": "Cross-Site Scripting (XSS) allows attackers to inject malicious scripts into web pag",
    "fix_suggestion": "Apply context-aware output encoding before rendering user content. Use your framework",
    "business_impact": "An attacker could steal session tokens from the admin panel, redirect users to phish",
    "attack_path": "UNKNOWN: deterministic fallback has no runtime proof.",
    "next_test": "Run one authorized, read-only validation test and stop if it changes state or exposes rea",
    "raw": {
      "pattern": "(render_template_string\\(|\\|safe\\b|innerHTML\\s*=|document\\.write\\(|dangerouslySetInner",
      "match": "|safe",
      "file": "C:\\Users\\Purushottam\\OneDrive\\Desktop\\CyberScurity Project - Copy\\Test\\vuln_bank_lab\\",
    }
  },
  {
    "id": "585f2b19-c895-48e3-91ac-747f9c101db6",

```

```

"title": "Idor in C:\\Users\\Purushottam\\OneDrive\\Desktop\\CyberScurity Project - Copy\\Test\\vuln_bank_lab\\",
"file": "C:\\Users\\Purushottam\\OneDrive\\Desktop\\CyberScurity Project - Copy\\Test\\vuln_bank_lab\\",
"line_start": 44,
"line_end": 44,
"severity": "MEDIUM",
"vuln_type": "IDOR",
"description": "Potential IDOR due to direct object reference.",
"rule_id": "fallback-idor",
"confidence": 0.8,
"cwe_id": "CWE-639",
"cvss_score": null,
"code_snippet": "      42 | @app.before_request\n      43 | def load_current_user():\n>>>      44 |     use\n"sources": [\n    "semgrep"\n],
"risk_score": 0.3936,
"risk_category": "LOW",
"category_source": "score",
"category_reason": "score-threshold",
"llm_explanation": "Insecure Direct Object Reference (IDOR) allows users to access other users' data by\n"fix_suggestion": "Add authorization checks that verify the requesting user owns or has permission to ac\n"business_impact": "An attacker could access or modify other users' session cookies by incrementing or g\n"attack_path": "UNKNOWN: deterministic fallback has no runtime proof.",
"next_test": "Run one authorized, read-only validation test and stop if it changes state or exposes rea\n"raw": {\n    "pattern": "(user_id\\s*=\\s*request\\.|SELECT\\s+\\*\\s+FROM\\s+users\\s+WHERE\\s+id\\s*=\\s*\\?|/ap\n    "match": "user_id = request.",\n    "file": "C:\\Users\\Purushottam\\OneDrive\\Desktop\\CyberScurity Project - Copy\\Test\\vuln_bank_lab\n}\n},\n{\n    "id": "367c86e3-e2b0-4ff8-83e2-5e6f0b3262a7",\n    "title": "Xss in C:\\Users\\Purushottam\\OneDrive\\Desktop\\CyberScurity Project - Copy\\Test\\vuln_bank\n    "file": "C:\\Users\\Purushottam\\OneDrive\\Desktop\\CyberScurity Project - Copy\\Test\\vuln_bank_lab\\",\n    "line_start": 24,\n    "line_end": 24,\n    "severity": "MEDIUM",\n    "vuln_type": "XSS",\n    "description": "Potential XSS via unsafe output handling.",\n    "rule_id": "fallback-xss",\n    "confidence": 0.9,\n    "cwe_id": "CWE-79",\n    "cvss_score": null,\n    "code_snippet": "      22 |     {% for tx in transactions %}\n      23 |         <tr>\n>>>      24 | \n"sources": [\n    "semgrep"\n],\n    "risk_score": 0.3626,\n    "risk_category": "LOW",\n    "category_source": "score",\n    "category_reason": "score-threshold",\n    "llm_explanation": "Cross-Site Scripting (XSS) allows attackers to inject malicious scripts into web pag\n    "fix_suggestion": "Apply context-aware output encoding before rendering user content. Use your framework\n    "business_impact": "An attacker could steal session tokens from user accounts and personal data, redirec\n    "attack_path": "UNKNOWN: deterministic fallback has no runtime proof.",\n    "next_test": "Run one authorized, read-only validation test and stop if it changes state or exposes rea\n    "raw": {\n        "pattern": "(render_template_string\\(|\\|safe\\b|innerHTML\\s*=|document\\.write\\(|dangerouslySetIn\n        "match": "|safe",\n        "file": "C:\\Users\\Purushottam\\OneDrive\\Desktop\\CyberScurity Project - Copy\\Test\\vuln_bank_lab\n    }\n},\n{\n    "id": "c067f94a-c836-4e2b-9b14-3e685c75ef88",\n    "title": "Xss in C:\\Users\\Purushottam\\OneDrive\\Desktop\\CyberScurity Project - Copy\\Test\\vuln_bank\n    "file": "C:\\Users\\Purushottam\\OneDrive\\Desktop\\CyberScurity Project - Copy\\Test\\vuln_bank_lab\\",\n    "line_start": 61,\n    "line_end": 61,\n    "severity": "MEDIUM",\n    "vuln_type": "XSS",\n    "description": "Potential XSS via unsafe output handling.",

```

```

"rule_id": "fallback-xss",
"confidence": 0.9,
"cwe_id": "CWE-79",
"cvss_score": null,
"code_snippet": "          59 |          <td>{{ tx.counterparty }}</td>\n          60 |          <td>\u20b9{{
"sources": [
    "semgrep"
],
"risk_score": 0.3626,
"risk_category": "LOW",
"category_source": "score",
"category_reason": "score-threshold",
"llm_explanation": "Cross-Site Scripting (XSS) allows attackers to inject malicious scripts into web pag
"fix_suggestion": "Apply context-aware output encoding before rendering user content. Use your framework
"business_impact": "An attacker could steal session tokens from user accounts and personal data, redirec
"attack_path": "UNKNOWN: deterministic fallback has no runtime proof.",
"next_test": "Run one authorized, read-only validation test and stop if it changes state or exposes rea
"raw": {
    "pattern": "(render_template_string\\(|\\|safe\\b|innerHTML\\s*=|document\\.write\\(|dangerouslySetInn
    "match": "|safe",
    "file": "C:\\Users\\Purushottam\\OneDrive\\Desktop\\CyberScurity Project - Copy\\Test\\vuln_bank_lab
}
},
{
    "id": "920f6864-5a0a-4354-a358-77481ef7d29a",
    "title": "Xss in C:\\Users\\Purushottam\\OneDrive\\Desktop\\CyberScurity Project - Copy\\Test\\vuln_bank_lab\\
    "file": "C:\\Users\\Purushottam\\OneDrive\\Desktop\\CyberScurity Project - Copy\\Test\\vuln_bank_lab\\
    "line_start": 7,
    "line_end": 7,
    "severity": "MEDIUM",
    "vuln_type": "XSS",
    "description": "Potential XSS via unsafe output handling.",
    "rule_id": "fallback-xss",
    "confidence": 0.9,
    "cwe_id": "CWE-79",
    "cvss_score": null,
    "code_snippet": "          5 |    <h1>Customer Notice Board</h1>\n          6 |    <p class=\"subtle\">This page
    "sources": [
        "semgrep"
    ],
    "risk_score": 0.3626,
    "risk_category": "LOW",
    "category_source": "score",
    "category_reason": "score-threshold",
    "llm_explanation": "Cross-Site Scripting (XSS) allows attackers to inject malicious scripts into web pag
    "fix_suggestion": "Apply context-aware output encoding before rendering user content. Use your framework
    "business_impact": "An attacker could steal session tokens from user accounts and personal data, redirec
    "attack_path": "UNKNOWN: deterministic fallback has no runtime proof.",
    "next_test": "Run one authorized, read-only validation test and stop if it changes state or exposes rea
    "raw": {
        "pattern": "(render_template_string\\(|\\|safe\\b|innerHTML\\s*=|document\\.write\\(|dangerouslySetInn
        "match": "|safe",
        "file": "C:\\Users\\Purushottam\\OneDrive\\Desktop\\CyberScurity Project - Copy\\Test\\vuln_bank_lab
    }
},
{
    "id": "9087f5b1-b070-45c2-bae9-ce9f3c08a434",
    "title": "Xss in C:\\Users\\Purushottam\\OneDrive\\Desktop\\CyberScurity Project - Copy\\Test\\vuln_bank_lab\\
    "file": "C:\\Users\\Purushottam\\OneDrive\\Desktop\\CyberScurity Project - Copy\\Test\\vuln_bank_lab\\
    "line_start": 23,
    "line_end": 23,
    "severity": "MEDIUM",
    "vuln_type": "XSS",
    "description": "Potential XSS via unsafe output handling.",
    "rule_id": "fallback-xss",
    "confidence": 0.9,
    "cwe_id": "CWE-79",
    "cvss_score": null,
    "code_snippet": "          21 |          <td>{{ ticket.name }}</td>\n          22 |          <td>{{ ticket.sub
    "sources": [
        "semgrep"
    ]
}

```

```

    ],
    "risk_score": 0.3626,
    "risk_category": "LOW",
    "category_source": "score",
    "category_reason": "score-threshold",
    "llm_explanation": "Cross-Site Scripting (XSS) allows attackers to inject malicious scripts into web pages.",
    "fix_suggestion": "Apply context-aware output encoding before rendering user content. Use your framework's built-in escaping functions.",
    "business_impact": "An attacker could steal session tokens from user accounts and personal data, redirecting them to malicious sites.",
    "attack_path": "UNKNOWN: deterministic fallback has no runtime proof.",
    "next_test": "Run one authorized, read-only validation test and stop if it changes state or exposes real data.",
    "raw": {
        "pattern": "(render_template_string\\(|\\|safe\\b|innerHTML\\s*=|document\\.write\\(|dangerouslySetInnerHTML\\s*=)",
        "match": "|safe",
        "file": "C:\\Users\\Purushottam\\OneDrive\\Desktop\\CyberScurity Project - Copy\\Test\\vuln_bank_lab\\vuln_bank_lab.py"
    }
},
{
    "id": "f9426289-4bcc-4afd-8660-f9e029b6a584",
    "title": "Open Redirect in C:\\Users\\Purushottam\\OneDrive\\Desktop\\CyberScurity Project - Copy\\Test\\vuln_bank_lab\\vuln_bank_lab.py",
    "file": "C:\\Users\\Purushottam\\OneDrive\\Desktop\\CyberScurity Project - Copy\\Test\\vuln_bank_lab\\vuln_bank_lab.py",
    "line_start": 88,
    "line_end": 88,
    "severity": "MEDIUM",
    "vuln_type": "OPEN_REDIRECT",
    "description": "Potential open redirect using attacker-controlled redirect target.",
    "rule_id": "fallback-open_redirect",
    "confidence": 0.8,
    "cwe_id": "CWE-601",
    "cvss_score": null,
    "code_snippet": "
86 | def logout():\n
87 |     next_url = request.args.get(\"next\", url_for('login'))\n
",
    "sources": [
        "semgrep"
    ],
    "risk_score": 0.3456,
    "risk_category": "LOW",
    "category_source": "score",
    "category_reason": "score-threshold",
    "llm_explanation": "Open redirect allows attackers to redirect users from your trusted domain to a malicious domain.",
    "fix_suggestion": "Validate redirect URLs against a whitelist of allowed domains. Use relative paths for redirects.",
    "business_impact": "Attackers could send users convincing phishing links via session cookies that appear to be from the legitimate site.",
    "attack_path": "UNKNOWN: deterministic fallback has no runtime proof.",
    "next_test": "Run one authorized, read-only validation test and stop if it changes state or exposes real data.",
    "raw": {
        "pattern": "(redirect\\(|\\.\\s*(?:request\\.\\.param|bar|next)|return\\s+redirect\\(|\\.\\.next)",
        "match": "redirect(next)",
        "file": "C:\\Users\\Purushottam\\OneDrive\\Desktop\\CyberScurity Project - Copy\\Test\\vuln_bank_lab\\vuln_bank_lab.py"
    }
}
]

```